Computer Programming 1

Laboratory 07

(functions and parameter passing – part 3)

Exercise 1: Scrabble

- Scrabble is a word game where players place letter tiles on a board to form words. Each letter has a value. A word's score is the sum of its letters' values.
- Write a program that computes a word's Scrabble score by summing the values of its letters.
- The letters are valued as follows:

Constraint: do not use arrays

Letter	Value
A, E, I, O, U, L, N, R, S, T	1
D, G	2
B, C, M, P	3
F, H, V, W, Y	4
K	5
J, X	8
Q, Z	10

For example, the word "cabbage" is worth 14 points:

- 3 points for C
- 1 point for A
- 3 points for B
- 3 points for B
- 1 point for A
- 2 points for G
- 1 point for E

Implementation constraints:

- Do not use any global variables
- Do not use arrays or any other complex data-types
- The user inserts the world char-by-char via standard input, till 0 is inserted
- Use at least one extra-function (with respect to the *main*) with parameter passing by reference/pointers
- Use c.in fail, ignore, and clean to check the stream

Exercise 1: Scrabble (2)

Variant:

- Each letter of the word in input is randomly generated by using the function rand() in the library cstdlib.
- Att: consider that only Letter (box below) takes points
- The letters are valued as follows:

Constraint: do not use arrays

Letter	Value
A, E, I, O, U, L, N, R, S, T	1
D, G	2
B, C, M, P	3
F, H, V, W, Y	4
K	5
J, X	8
Q, Z	10

For example, the word "cabbage" is worth 14 points:

- 3 points for C
- 1 point for A
- 3 points for B
- 3 points for B
- 1 point for A
- 2 points for G
- 1 point for E

Implementation constraints:

- Do not use any global variables
- Do not use arrays or any other complex data-types
- The user inserts the world char-by-char via standard input, till 0 is inserted
- Use at least one extra-function (with respect to the *main*) with parameter passing by reference/pointers
- Use c.in fail, ignore, and clean to check the stream

Exercise 2: Dungeons & Dragons (D&D)

- For the Dungeons & Dragons game, each player starts by generating a character they can play with. This character has, among other things, six abilities; strength, dexterity, constitution, intelligence, wisdom and charisma. These six abilities have scores that are determined randomly. You do this by rolling four 6-sided dice and recording the sum of the largest three dice. You do this six times, once for each ability.
- Your character's initial hitpoints are 10 + your character's constitution modifier. You find your character's constitution modifier by subtracting 10 from your character's constitution, divide by 2 and round down.
- Task: write a random character generator that follows the above rules and generates an output like the one of the example below.

For example, the six throws of four dice may look like:
5, 3, 1, 6: strength 5 + 3 + 6 = 14
3, 2, 5, 3: dexterity 3 + 5 + 3 = 11
1, 1, 1, 1: constitution 1 + 1 + 1 = 3
2, 1, 6, 6: intelligence 2 + 6 + 6 = 14
3, 5, 3, 4: wisdom 5 + 3 + 4 = 12.
6, 6, 6, 6: charisma 6 + 6 + 6 = 18
Because constitution is 3, the constitution modifier is -4 and the hitpoints are 6.

Constraint: do not use arrays

Implementation constraints:

- Do not use any global variables
- Do not use arrays or any other complex data structure
- Do not use any libraries (apart iostream and cstdlib)
- Use at least two additional functions (apart the a *main*) named:
  1. ability (to define the value of the six abilities)
  2. modifierComputation (to calculate the hitpoints)

Exercise 3: Abundant Numbers

- Write a program that finds the **Abundant Numbers** between two positive numbers inserted by the user (e.g., 1 and 1000).

Definition An abundant number is a positive integer for which the sum of its proper divisors (divisors excluding the number itself) is greater than the number itself.

For example, 12 is abundant because its proper divisors (1, 2, 3, 4, and 6) sum to 16, which is greater than 12.

E.g., The first few abundant numbers are 12, 18, 20, 24, 30, and 36.

Implementation constraints:

- Do not use any global variables
- Do not use arrays or any other complex data-types
- Do not use other libraries apart iostream and cmath
- The user inserts the two numbers via standard input, till 0 is inserted
- Use c.in fail, ignore, and clean to check the stream
- Use at least two extra-functions (with respect to the *main*)

Constraint: do not use arrays